



```
In [1]: # Import Libraries
# -----
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from statsmodels.tsa.arima.model import ARIMA
import os
```

```
In [2]: # Load The Dataset
# -----
temp_df=pd.read_csv("C:/Sure Proed/Climate Analytics/SQL/temperature.csv")
co2_df=pd.read_csv("C:/Sure Proed/Climate Analytics/SQL/co2_emissions.csv")
rain_df =pd.read_csv("C:/Sure Proed/Climate Analytics/SQL/rainfall.csv")
```

```
In [3]: # Merge datasets into single DataFrame
# -----
df_merged =temp_df.merge(co2_df, on=["year", "region","country"],how="inner").
df_merged
```

```
Out[3]:
```

	year	region	country	Temp_Anomaly °C	CO2_Emissions (MtCO2)	rainfall_mm
0	2000	South America	Argentina	-0.03	187.7	522.5
1	2000	Oceania	Australia	0.11	346.0	433.2
2	2000	South America	Brazil	0.10	364.6	2180.6
3	2000	North America	Canada	0.19	554.4	498.2
4	2000	Asia	China	0.49	3249.4	669.5
...	...	...	...	...	...	...
515	2025	Africa	South Africa	1.25	417.5	437.9
516	2025	Europe	Spain	1.75	251.2	582.6
517	2025	Europe	Turkey	1.85	427.0	584.9
518	2025	Europe	UK	1.48	356.2	990.0
519	2025	North America	USA	1.67	4682.5	678.6

520 rows × 6 columns

```
In [4]: # Handle missing values
```

```
#-----  
df_merged.isnull().sum()
```

```
Out[4]: year          0  
       region        0  
       country       0  
       Temp_Anomaly °C)  0  
       CO2_Emissions (MtCO2)  0  
       rainfall_mm    0  
       dtype: int64
```

```
In [5]: df_merged.duplicated().sum()
```

```
Out[5]: np.int64(0)
```

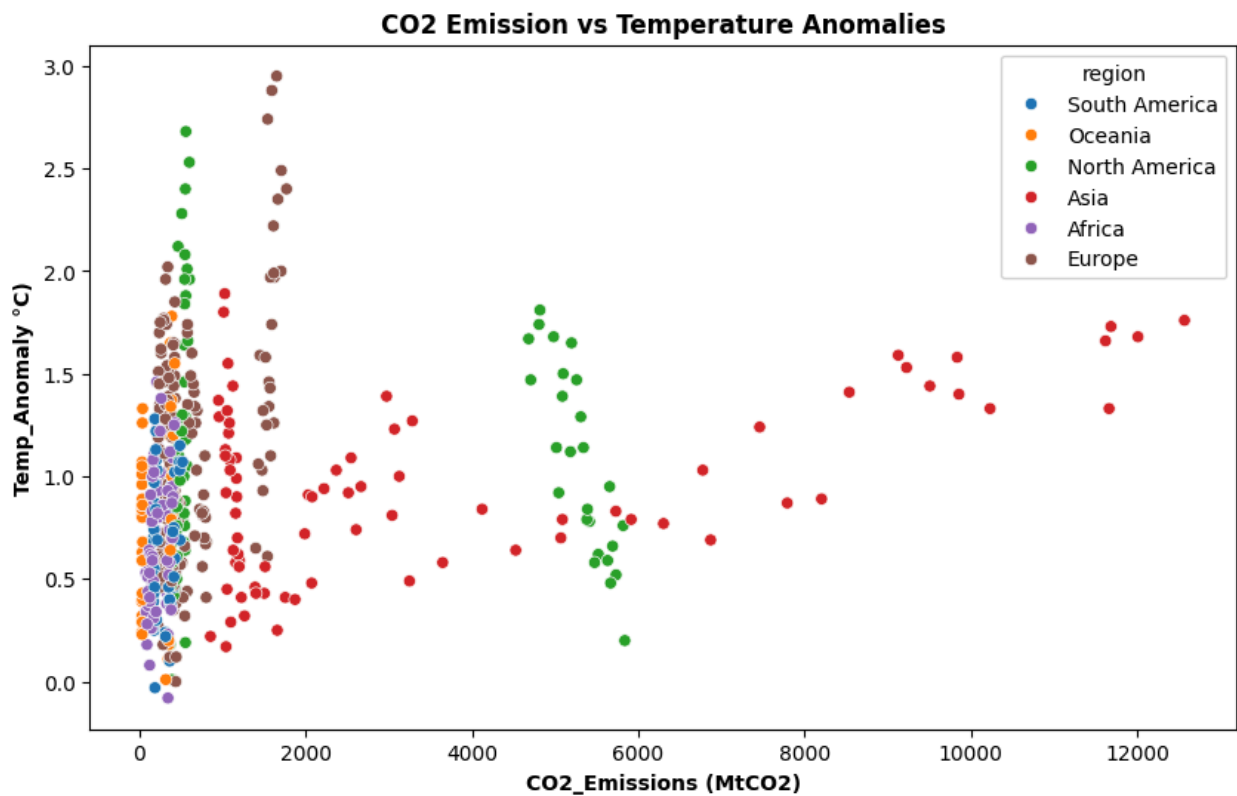
```
In [6]: df_merged.dtypes
```

```
Out[6]: year          int64  
       region        object  
       country       object  
       Temp_Anomaly °C)  float64  
       CO2_Emissions (MtCO2)  float64  
       rainfall_mm    float64  
       dtype: object
```

```
In [7]: print(type(df_merged))
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
In [8]: # Scatter plots – anomaly vs emissions  
# -----  
import seaborn as sns  
import matplotlib.pyplot as plt  
  
plt.figure(figsize=(10,6))  
  
sns.scatterplot(  
    data=df_merged,  
    x="CO2_Emissions (MtCO2)",  
    y="Temp_Anomaly °C)",  
    hue="region"  
)  
  
plt.title("CO2 Emission vs Temperature Anomalies", fontweight="bold")  
plt.xlabel("CO2_Emissions (MtCO2)", fontweight="bold")  
plt.ylabel("Temp_Anomaly °C)", fontweight="bold")  
  
plt.show()
```



```
In [9]: # Group by for continent summaries
# -----

# Grouping data by region and calculating mean to study overall climate trend
# -----
continent_summary= df_merged.groupby("region").agg({"CO2_Emissions (MtCO2)": "mean",
                                                    "Temp_Anomaly °C": "mean",
                                                    "rainfall_mm": "mean"
                                                    }).round(2)

print(continent_summary)
```

region	CO2_Emissions (MtCO2)	Temp_Anomaly °C)	rainfall_mm
Africa	227.90	0.68	610.93
Asia	3708.59	0.96	1113.10
Europe	575.38	1.11	710.38
North America	2095.21	1.10	635.99
Oceania	204.04	0.80	945.81
South America	292.64	0.68	1336.86

```
In [10]: # Regional Carbon Emission Comparison(Bar chart)
# -----

plt.figure(figsize=(10,6))

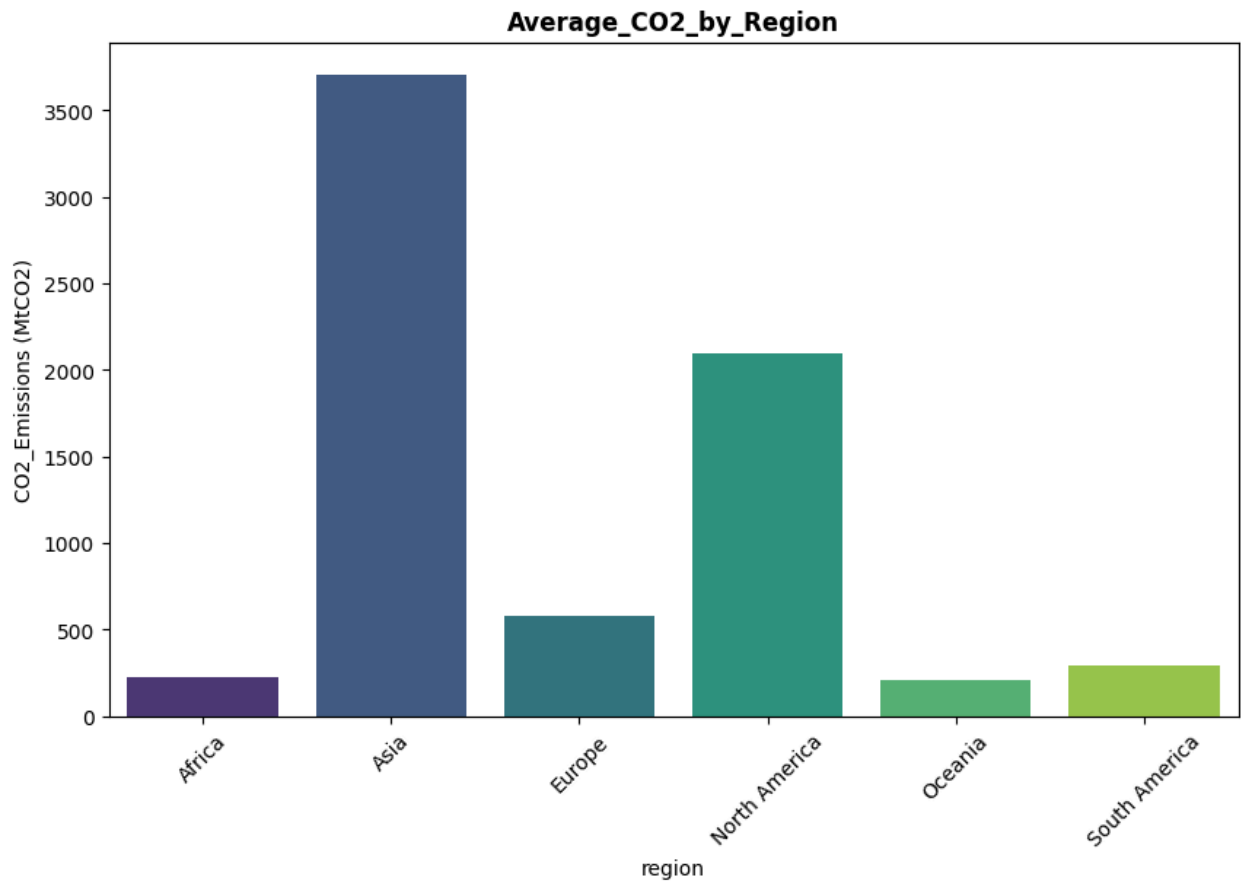
sns.barplot(
```

```

data=continent_summary.reset_index(),
x="region",
y="CO2_Emissions (MtCO2)",hue="region",palette="viridis",legend=False
)

plt.title("Average_CO2_by_Region",fontweight='black')
plt.xticks(rotation=45)
plt.show()

```



```

In [11]: ## Calculating maximum and minimum temperature anomalies to understand extreme
# -----

```

```

continent_summary.groupby("region")["Temp_Anomaly °C"].max()
continent_summary.groupby("region")["Temp_Anomaly °C"].min()

```

```

Out[11]: region
Africa          0.68
Asia            0.96
Europe          1.11
North America   1.10
Oceania         0.80
South America   0.68
Name: Temp_Anomaly °C, dtype: float64

```

```

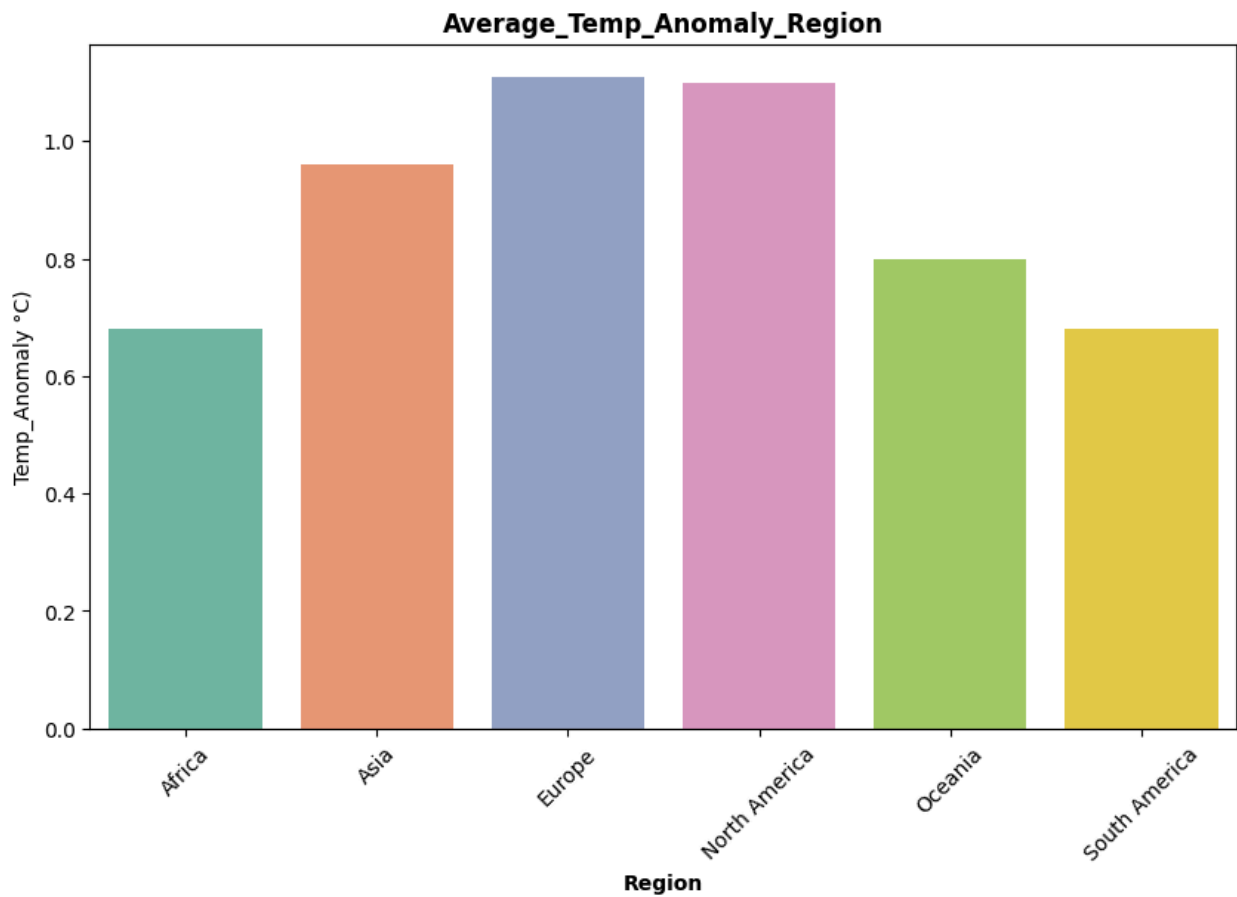
In [12]: # Regional Climate Temperature Changes(Bar Chart)
# -----

```

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=continent_summary.reset_index(),
    x="region",
    y="Temp_Anomaly °C",hue="region",palette="Set2",legend=False,
)

plt.title("Average_Temp_Anomaly_Region",fontweight='black')
plt.xlabel("Region",fontweight='bold')
plt.xticks(rotation=45)
plt.show()
```



```
In [13]: # Region-wise Median Climate Summary
# -----
continent_summary.groupby("region")[["CO2_Emissions (MtCO2)", "Temp_Anomaly °C"]
```

Out[13]:

	CO2_Emissions (MtCO2)	Temp_Anomaly °C	rainfall_mm
region			
Africa	227.90	0.68	610.93
Asia	3708.59	0.96	1113.10
Europe	575.38	1.11	710.38
North America	2095.21	1.10	635.99
Oceania	204.04	0.80	945.81
South America	292.64	0.68	1336.86

region			
Africa	227.90	0.68	610.93
Asia	3708.59	0.96	1113.10
Europe	575.38	1.11	710.38
North America	2095.21	1.10	635.99
Oceania	204.04	0.80	945.81
South America	292.64	0.68	1336.86

In [14]: *## Identifying extreme rainfall conditions (highest and lowest values) in the*
#-----
continent_summary.groupby("region")["rainfall_mm"].max()
continent_summary.groupby("region")["rainfall_mm"].min()

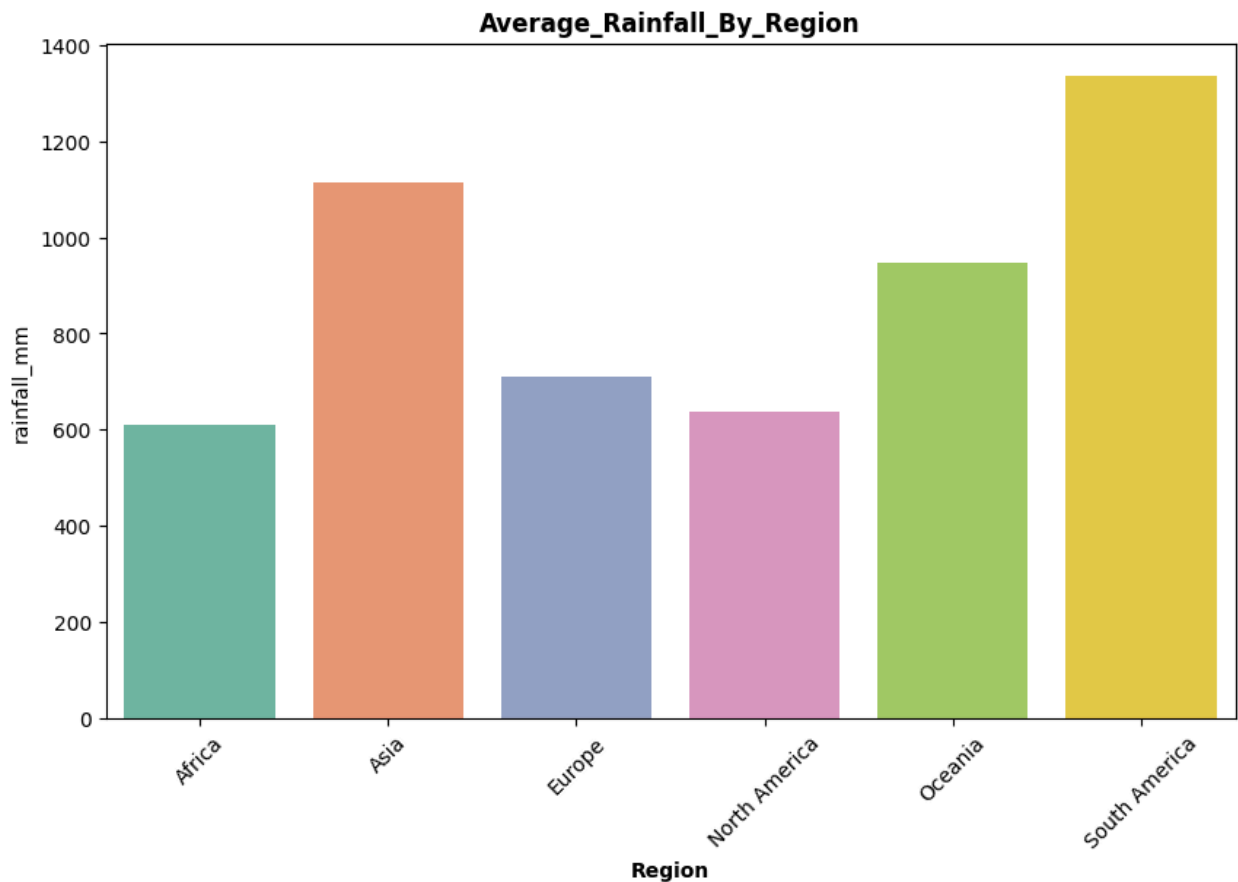
Out[14]: region
Africa 610.93
Asia 1113.10
Europe 710.38
North America 635.99
Oceania 945.81
South America 1336.86
Name: rainfall_mm, dtype: float64

In [15]: *# Average Rainfall Distribution by Region(Bar_Chart)*

plt.figure(figsize=(10,6))

sns.barplot(
 data=continent_summary.reset_index(),
 x="region",
 y="rainfall_mm",hue="region",palette="Set2",legend=False,
)

plt.title("Average_Rainfall_By_Region",fontweight='black')
plt.xlabel("Region",fontweight='bold')
plt.xticks(rotation=45)
plt.show()



```
In [33]: # Linear Regression – CO2 → Temperature
# -----

# Features
X= df_merged[["CO2_Emissions (MtCO2)","rainfall_mm","year"]].values # features

# Target
y= df_merged[["Temp_Anomaly °C")]].values # target

# Train Test Split # 80% train, 20% test
X_train,X_test,y_train,y_test=train_test_split(X,y, test_size=0.2,random_state=42)

# Creating Model
model=LinearRegression()

# Train Model
model.fit(X_train,y_train) # train the model
```

```
Out[33]: ▼ LinearRegression ⓘ ⓘ
LinearRegression()
```

```
In [34]: y_pred = model.predict(X_test) # R2 = 0.004 means CO2
print("R2 Score:", r2_score(y_test, y_pred))
```

R2 Score: 0.626720352621791

```
In [36]: print("Slope:", model.coef_)
         print("Intercept:", model.intercept_)
```

```
Slope: [[ 3.18967942e-05 -1.19819140e-04  4.66849108e-02]]
Intercept: [-92.9470528]
```

```
In [19]: print(model.predict(X_test[:5]))
```

```
[[0.96467527]
 [0.80832142]
 [0.92042106]
 [0.95421919]
 [0.85897806]]
```

```
In [68]: # Time-series forecast using ARIMA.
         # -----

         # Load dataset
         df = pd.read_csv("C:/Sure Proed/Climate Analytics/SQL/co2_emissions.csv")

         # Aggregate emissions by year using average
         yearly = df.groupby("year")["CO2_Emissions (MtCO2)"].mean()

         # Fit ARIMA model (order can be tuned based on AIC/BIC)
         model = ARIMA(yearly, order=(2,1,2))
         model_fit = model.fit()

         # Forecast for 2026–2030
         forecast = model_fit.forecast(steps=5)

         for year,value in zip(forecast_years,forecast):
             print(f"{year}: {value:.2f}")

         # Plot historical + forecast
         plt.figure(figsize=(12,6))
         plt.plot(yearly.index, yearly.values, label="Historical", color="blue")
         plt.plot(range(2026, 2031), forecast, label="ARIMA Forecast", color="red", mar

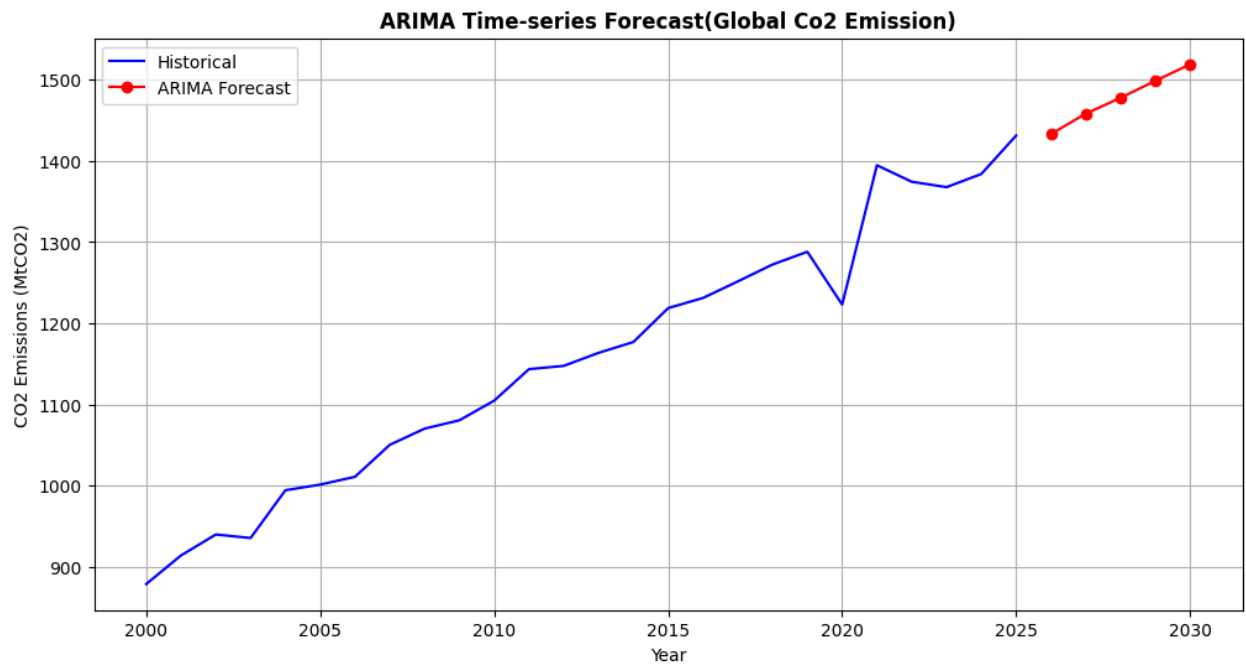
         plt.title("ARIMA Time-series Forecast(Global Co2 Emission)",fontweight="bold")
         plt.xlabel("Year")
         plt.ylabel("CO2 Emissions (MtCO2)")
         plt.legend()
         plt.grid(True)
         plt.show()
```



```

C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:4
73: ValueWarning: An unsupported index was provided. As a result, forecasts can
not be generated. To use the model for forecasting, use one of the supported cl
asses of index.
    self._init_dates(dates, freq)
C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:4
73: ValueWarning: An unsupported index was provided. As a result, forecasts can
not be generated. To use the model for forecasting, use one of the supported cl
asses of index.
    self._init_dates(dates, freq)
C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:4
73: ValueWarning: An unsupported index was provided. As a result, forecasts can
not be generated. To use the model for forecasting, use one of the supported cl
asses of index.
    self._init_dates(dates, freq)
C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\tsa\statespace\sarima
x.py:966: UserWarning: Non-stationary starting autoregressive parameters found.
Using zeros as starting parameters.
    warn('Non-stationary starting autoregressive parameters'
C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\tsa\statespace\sarima
x.py:978: UserWarning: Non-invertible starting MA parameters found. Using zeros
as starting parameters.
    warn('Non-invertible starting MA parameters found.')
C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\base\model.py:607: Conv
ergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_r
etvals
    warnings.warn("Maximum Likelihood optimization failed to "
C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:8
37: ValueWarning: No supported index is available. Prediction results will be g
iven with an integer index beginning at `start`.
    return get_prediction_index(
C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:8
37: FutureWarning: No supported index is available. In the next version, callin
g this method in a model without a supported index will result in an exception.
    return get_prediction_index(
2026: 1432.99
2027: 1457.99
2028: 1477.53
2029: 1498.35
2030: 1518.87

```



```
In [61]: # Load dataset
df = pd.read_csv("C:/Sure Proed/Climate Analytics/SQL/temperature.csv")

df.set_index("year", inplace=True)

# Aggregate emissions by year using average
yearly = df.groupby("year")["Temp_Anomaly °C"].mean()

# Fit ARIMA model (order can be tuned based on AIC/BIC)
model = ARIMA(yearly, order=(2, 1, 2))
model_fit = model.fit()

# Forecast for 2026–2030
forecast = model_fit.forecast(steps=5)

for year, value in zip(forecast_years, forecast):
    print(f"{year}: {value:.2f} °C")

# Plot historical + forecast
plt.figure(figsize=(12,6))
plt.plot(yearly.index, yearly.values, label="Historical", color="Green")
plt.plot(range(2026, 2031), forecast, label="ARIMA Forecast", color="red", mar

plt.title("ARIMA Time-series Forecast(Global Temp Anomaly)", fontweight="bold")
plt.xlabel("Year")
plt.ylabel("Temp_Anomaly °C")
plt.legend()
plt.grid(True)
```

```
plt.show()
```

```
C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: An unsupported index was provided. As a result, forecasts can not be generated. To use the model for forecasting, use one of the supported classes of index.
```

```
self._init_dates(dates, freq)
```

```
C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: An unsupported index was provided. As a result, forecasts can not be generated. To use the model for forecasting, use one of the supported classes of index.
```

```
self._init_dates(dates, freq)
```

```
C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: An unsupported index was provided. As a result, forecasts can not be generated. To use the model for forecasting, use one of the supported classes of index.
```

```
self._init_dates(dates, freq)
```

```
C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:837: ValueWarning: No supported index is available. Prediction results will be given with an integer index beginning at `start`.
```

```
return get_prediction_index()
```

```
C:\Users\LENOVO\anaconda3\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:837: FutureWarning: No supported index is available. In the next version, calling this method in a model without a supported index will result in an exception.
```

```
return get_prediction_index()
```

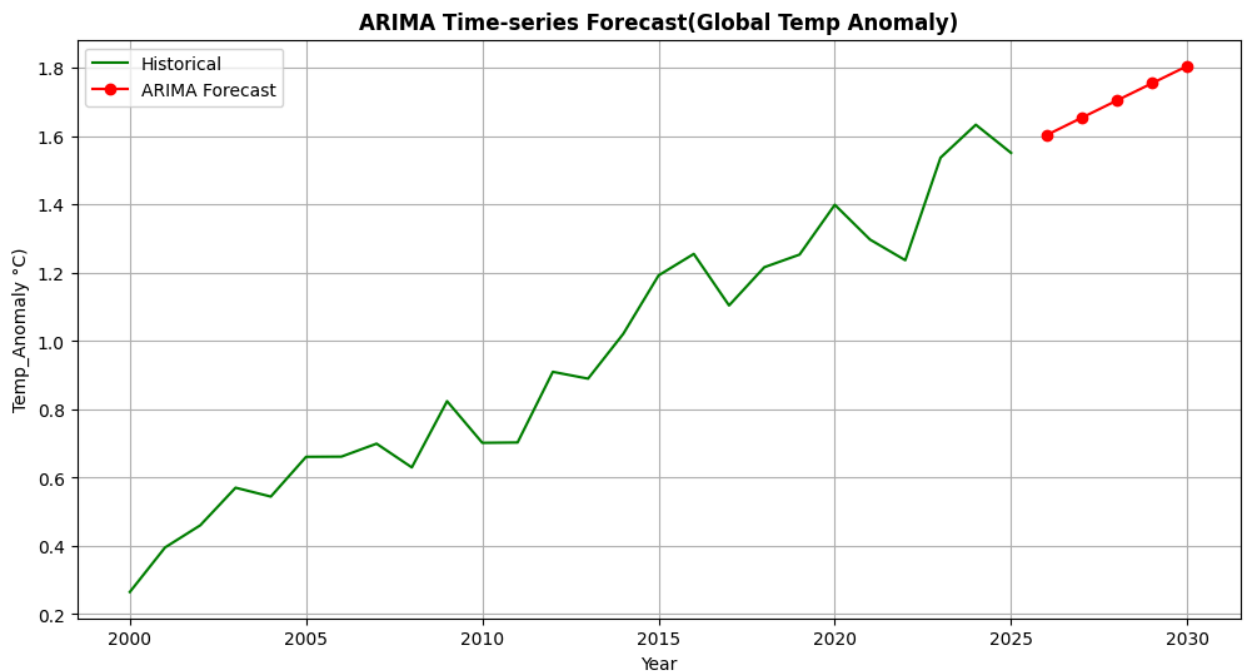
2026: 1.60 °C

2027: 1.65 °C

2028: 1.70 °C

2029: 1.76 °C

2030: 1.81 °C



```
In [38]: # Heatmap for rainfall anomalies.  
# -----
```

```

# Create pivot table for heatmap
# Rows = Country, Columns=Year, Values= Rainfall Anomaly

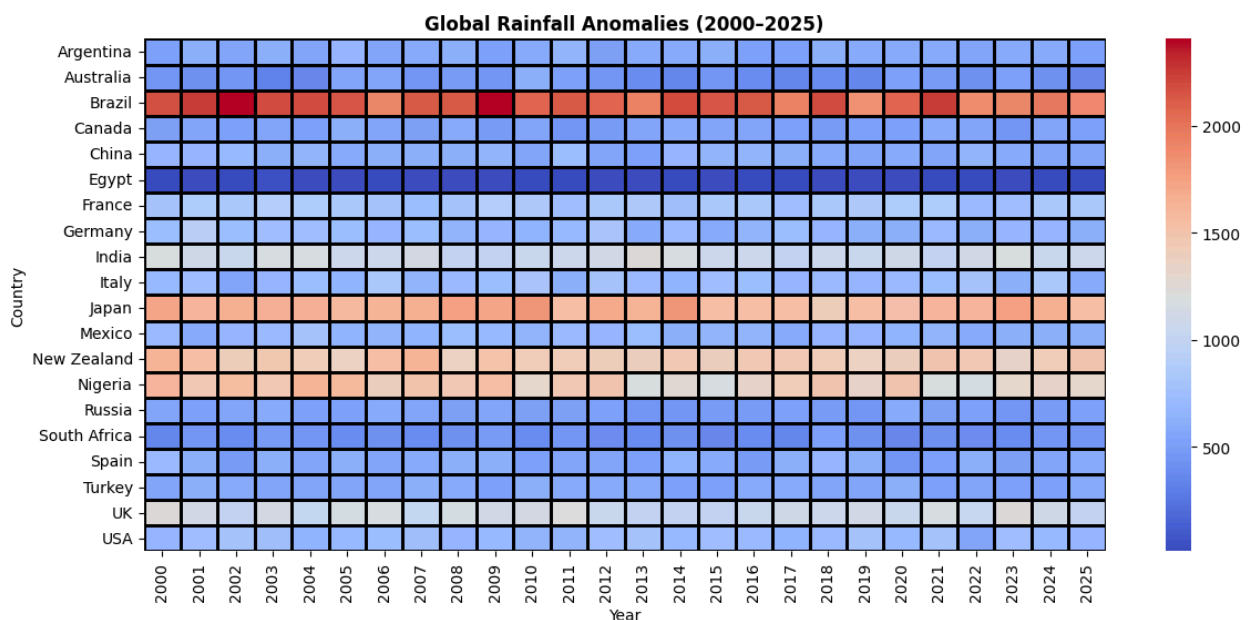
Heatmap_data=df_merged.pivot_table(values='rainfall_mm',index='country',column

# Heatmap Chart
# -----

plt.figure(figsize=(14,6))
sns.heatmap(Heatmap_data,cmap='coolwarm',linewidths=1,linecolor="black")

plt.title("Global Rainfall Anomalies (2000–2025)",fontweight='black')
plt.xlabel("Year")
plt.ylabel("Country")
plt.show()

```



```

In [39]: # Automated Data Pipeline using Functions
# -----

# 1. Load Data
def load_data():
    temp=pd.read_csv("C:/Sure Proed/Climate Analytics/SQL/temperature.csv")
    co2=pd.read_csv("C:/Sure Proed/Climate Analytics/SQL/co2_emissions.csv")
    rainfall =pd.read_csv("C:/Sure Proed/Climate Analytics/SQL/rainfall.csv")
    return co2, rainfall, temp

# 2. Clean Data
def clean_data(df,numeric_cols):
    # Convert numeric columns to float
    for col in numeric_cols:
        df[col]=pd.to_numeric(df[col],errors='coerce')
    # Drop missing values
    df=df.dropna()

```

```

        return df

# 3. Merge Data
def merge_data(co2, rainfall, temp):
    # Merge on Year + Region
    merged = pd.merge(co2, rainfall, on=["year", "region"], how="inner")
    merged = pd.merge(merged, temp, on=["year", "region"], how="inner")
    return merged

# 4. Feature Engineering
def calculate_features(df):
    # Rainfall anomaly index (deviation from mean)
    df["Rainfall_Anomaly_Index"] = df.groupby("region")["rainfall_mm"].transform(lambda x: (x - x.mean()) / x.std())
    # CO2 growth rate
    df["CO2_Growth_Rate"] = df.groupby("region")["CO2_Emissions (MtCO2)"].pct_change()
    return df

# 5. Analysis Functions
def analyze_trends(df):
    # Correlation between CO2 and Temp anomaly
    corr = df["CO2_Emissions (MtCO2)"].corr(df["Temp_Anomaly °C"])
    print("Correlation between CO2 and Temperature anomaly", corr)

def visualization_results(df):
    # Line chart for CO2 emissions
    plt.figure(figsize=(8, 6))
    sns.lineplot(data=df, x="year", y="CO2_Emissions (MtCO2)", hue="region")
    plt.title("CO2 Emissions Trend (2000-2025)", fontweight='bold')
    plt.show()

# 6. Run Pipeline
def run_pipeline():
    co2, rainfall, temp = load_data()
    co2 = clean_data(co2, ["CO2_Emissions (MtCO2)"])
    rainfall = clean_data(rainfall, ["rainfall_mm"])
    temp = clean_data(temp, ["Temp_Anomaly °C"])

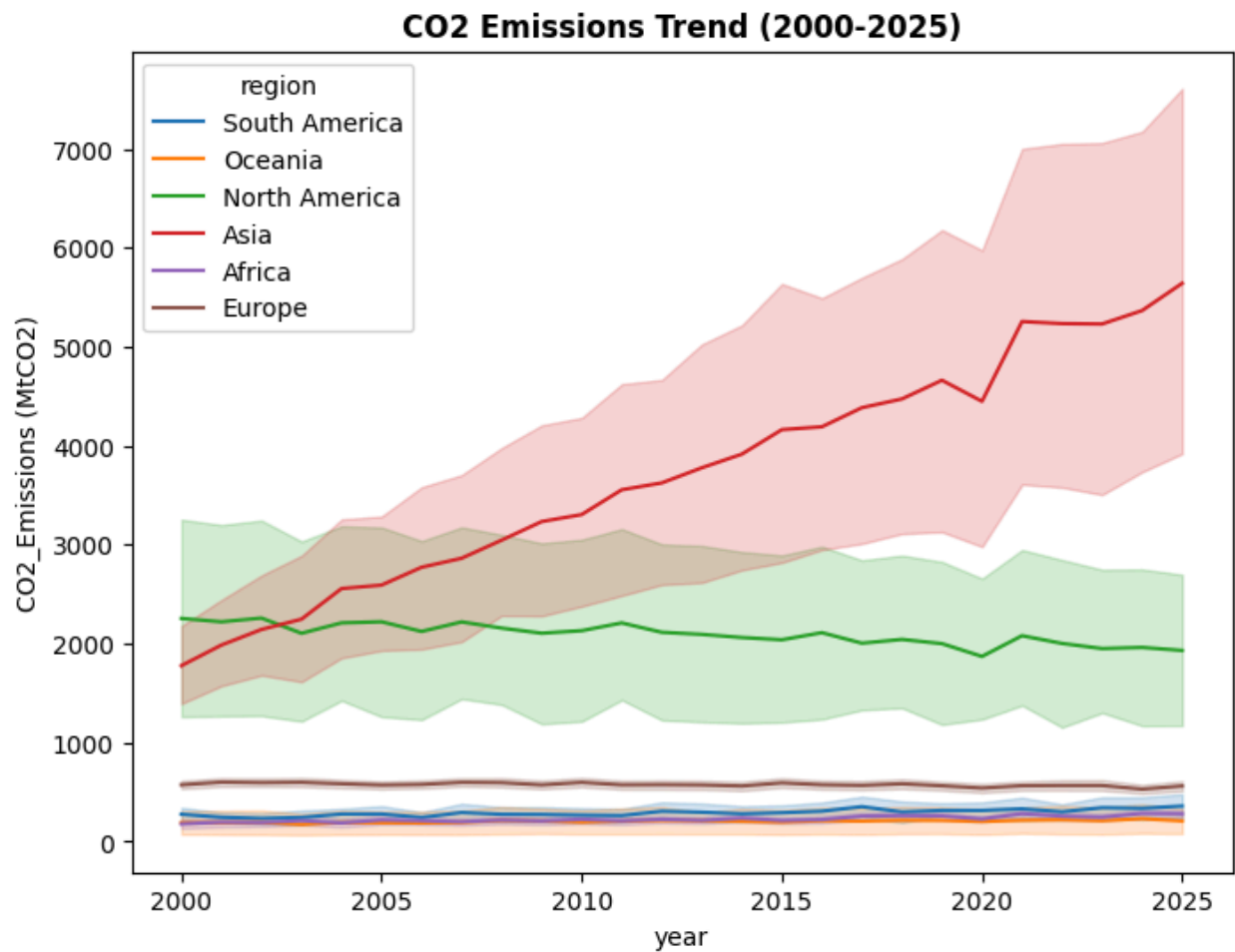
    merged = merge_data(co2, rainfall, temp)
    merged = calculate_features(merged)

    analyze_trends(merged)
    visualization_results(merged)
    return merged

final_df = run_pipeline()

```

Correlation between CO2 and Temperature anomaly 0.02012868645827149



```
In [75]: # Exporting the Processed Dataset
# -----
df_merged.to_csv("Processed_Climate_analytics.csv",index=False)
```

```
In [74]: import os
print(os.getcwd())
```

C:\Users\LENOVO

```
In [ ]:
```